

Topic: Operations on Singly Linked List

Q. Describe the fundamental operations performed on a Singly Linked List: Traversal, Insertion, Deletion, and Searching, detailing the logic behind each.

★ Definition to Remember

Manipulating a Singly Linked List requires careful **pointer management**. Because access is strictly sequential, we must use temporary pointers (`temp`) to traverse the list and update the `next` links without breaking the chain or losing memory references.

1. Traversal & Searching

Traversal (Printing the list):

Visit every node from `head` to `NULL`.

* **Logic:** `temp = head`. While `temp != NULL`, print `temp->data`, and move forward using `temp = temp->next`.

Searching:

* **Logic:** Same as traversal, but compare `temp->data` to the target value. Return if found; if `temp` reaches `NULL`, the value doesn't exist.

2. Insertion Operations

Position	Logic Steps
At Beginning	1. Create <code>new_node</code>. 2. Set <code>new_node->next = head</code>. 3. Update <code>head = new_node</code>.
At End	1. Create <code>new_node</code> (<code>next = NULL</code>). 2. Traverse <code>temp</code> to the last node. 3. Set <code>temp->next = new_node</code>.
At Specific Position	1. Traverse <code>temp</code> to the node <i>before</i> the target position. 2. Set <code>new_node->next = temp->next</code>. 3. Set <code>temp->next = new_node</code>.

3. Deletion Operations

(Always check for Underflow: if `head == NULL`, list is empty).

Position	Logic Steps
From Beginning	1. <code>temp = head</code> . 2. Update <code>head = head→next</code> . 3. <code>free(temp)</code> .
From End	1. Traverse to the second-to-last node. 2. <code>free(temp→next)</code> . 3. Set <code>temp→next = NULL</code> .
Specific Node	1. Traverse <code>temp</code> to the node <i>before</i> the target. 2. Bypass target: <code>temp→next = target→next</code> . 3. <code>free(target)</code> .

4. Critical Edge Cases

- **Empty List:** Trying to delete causes underflow.
- **Single Node List:** Deleting it means updating `head` to `NULL` .
- **Lost Pointers:** If you update `temp→next` before linking the new node, the rest of the list is permanently lost (Memory Leak).

★ Must-Write Points (for 10 marks)

1. Fundamental operations: **Traversal, Searching, Insertion, Deletion.**
2. **Traversal:** `temp = head` ; loop while `temp ≠ NULL` ; move using `temp = temp→next` .
3. **Insert at Beginning:** `new_node→next = head` ; `head = new_node` ; ($O(1)$ time).
4. **Insert at End:** Traverse to last node, link it to `new_node` ($O(n)$ time).
5. **Delete from Beginning:** move `head` to `head→next` , then `free()` the old head.
6. **Delete from Middle/End:** traverse to the node *before* the target, bypass the target, then `free()` it.
7. Always handle edge cases: empty list (underflow) and careful pointer sequence to avoid losing the list.

⚡ Quick Recall

Traversal (`temp=temp→next`) → Insert Begin ($O(1)$, update head) → Insert End ($O(n)$, traverse to last) → Delete (Bypass node, free memory) → Watch out for memory leaks!